

Robot Teknolojisine Giriş BLM4830



Öğr. Grv. Furkan ÇAKMAK

Ders Tanıtım Formu ve Konular

BLM4830
Robot
Teknolojisine
Giriş
Hafta 7

Hafta	Tarih	Konular
1	2.03.2022	Ders Tanıtımı, ROS ve Platform Tanıtımı, Robot Çeşitleri ve Robotik Konuları Başlangıcı
2	9.03.2022	Kinematik - Genel Tanımlar - Diferansiyel Sürürlü Robot İçin Hesaplama Örnekleri
3	16.03.2022	Sensörler - Çeşitleri ve Çalışma Sistematiikleri ve Uygulamaları
4	23.03.2022	Odometri ve Lokalizasyon Kavramları
5	30.03.2022	Haritalama Yöntemleri ve Uygulamaları
6	6.04.2022	Uygulama 1 (Laboratuvar) (Ödev İlanı)
7	13.04.2022	Navigasyon ve Keşif Yaklaşımları ve Uygulamaları
8	20.04.2022	Ara Sınav
9	27.04.2022	Uygulama 2 (Laboratuvar) (Ödev Teslimi)
10	4.05.2022	Tatil - Ramazan Bayramı Arifesi
11	11.05.2022	Robot Üzerinden Görüntü İşleme Teknikleri
12	18.05.2022	Robot Üzerinden Görüntü İşleme Teknikleri (Devam)
13	25.05.2022	3B Haritalama Yöntemleri
14	1.06.2022	Proje Sunumları

Öğr. Grv. Furkan ÇAKMAK

Robot Navigation

- One of the most basic things that a robot can do is to move around the world.
- To do this effectively, the robot needs to know where it is and where it should be going
- This is usually achieved by giving the robot a **map** of the world, a **starting location**, and a **goal location**
- We'll look at how to make your robot autonomously navigate from one part of the world to another, using the map and the ROS navigation packages

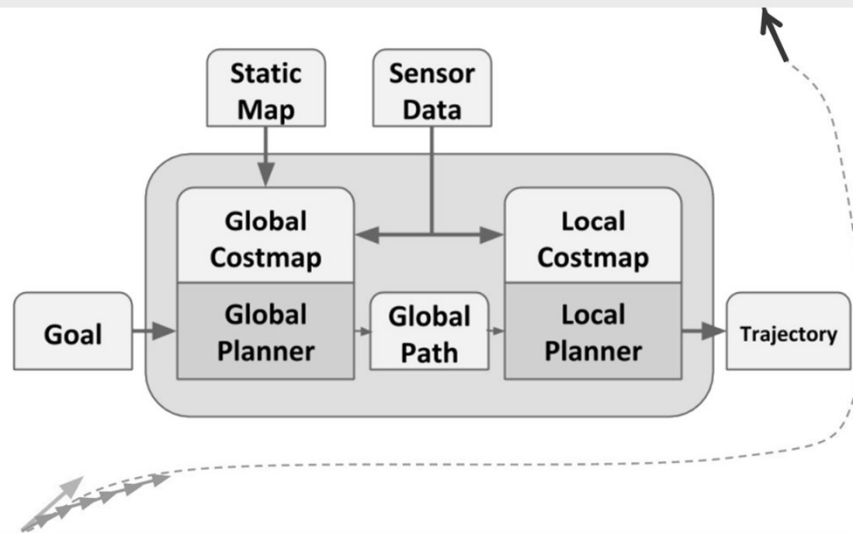
(C)2016 Roi Yehoshua

ROS Navigation Stack

- <http://wiki.ros.org/navigation>
- The goal of the navigation stack is to move a robot from one position to another position safely (without crashing or getting lost)
- It takes in information from the odometry and sensors, and a goal pose and outputs safe velocity commands that are sent to the robot
- [ROS Navigation Introductory Video](#)

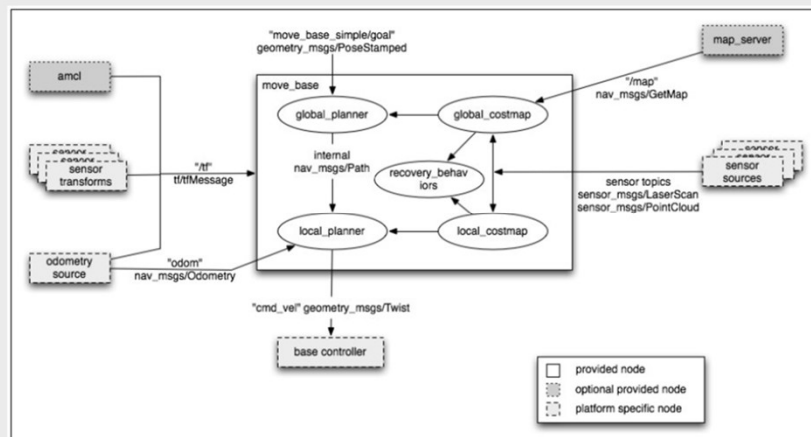
(C)2016 Roi Yehoshua

Overview of ROS Navigation



(C)2016 Roi Yehoshua

ROS Navigation Stack



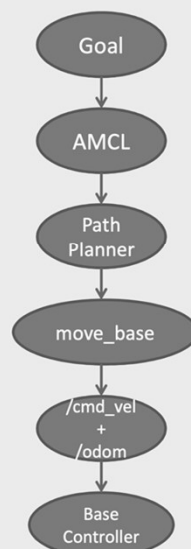
(C)2016 Roi Yehoshua

Navigation Stack Main Components

Package/Component	Description
map_server	offers map data as a ROS Service
gmapping	provides laser-based SLAM
amcl	a probabilistic localization system
global_planner	implementation of a fast global planner for navigation
local_planner	implementations of the Trajectory Rollout and Dynamic Window approaches to local robot navigation
move_base	links together the global and local planner to accomplish the navigation task

(C)2016 Roi Yehoshua

Navigation Main Steps



(C)2016 Roi Yehoshua

Install Navigation Stack

- The navigation stack is not part of the standard ROS Indigo installation
- To install the navigation stack type:

```
$ sudo apt-get install ros-indigo-navigation
```

(C)2016 Roi Yehoshua

Navigation Stack Requirements

Three main hardware requirements

- The navigation stack can only handle a **differential drive and holonomic wheeled robots**
 - It can also do certain things with biped robots, such as localization, as long as the robot does not move sideways
- A planar **laser** must be mounted on the mobile base of the robot to create the map and localization
 - Alternatively, you can generate something equivalent to laser scans from other sensors (**Kinect** for example)
- Its performance will be **best on robots that are nearly square or circular**

(C)2016 Roi Yehoshua

Navigation Planners

- Our robot will move through the map using two types of navigation—global and local
- The **global planner** is used to create **paths for a goal** in the map or a far-off distance
- The **local planner** is used to create **paths in the nearby distances** and avoid obstacles

(C)2016 Roi Yehoshua

Global Planner

- NavFn provides a fast interpolated navigation function that creates plans for a mobile base
- The **global plan** is computed **before the robot starts moving toward the next destination**
- The planner **operates on a costmap** to find a minimum cost plan from a start point to an end point in a grid, using **Dijkstra's algorithm**
- The global planner **generates a series of waypoints** for the local planner to follow

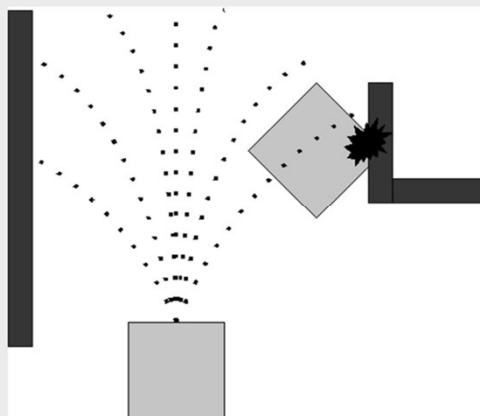
(C)2016 Roi Yehoshua

Local Planner

- **Chooses appropriate velocity commands** for the robot to traverse **the current segment of the global path**
- Combines sensory and odometry data with both global and local cost maps
- **Can recompute the robot's path on the fly** to keep the robot from striking objects yet still allowing it to reach its destination
- Implements the **Trajectory Rollout and Dynamic Window** algorithm

(C)2016 Roi Yehoshua

Trajectory Rollout Algorithm



Taken from ROS Wiki http://wiki.ros.org/base_local_planner

(C)2016 Roi Yehoshua

Trajectory Rollout Algorithm

1. Discretely **sample in the robot's control space ($dx, dy, d\theta$)**
2. For each sampled velocity, perform forward simulation from the robot's current state to **predict what would happen if the sampled velocity were applied for some (short) period of time**
3. **Evaluate each trajectory** resulting from the forward simulation, **using a metric** that incorporates characteristics such as: proximity to obstacles, proximity to the goal, proximity to the global path, and speed
4. **Discard illegal trajectories** (those that collide with obstacles)
5. **Pick the highest-scoring trajectory** and send the associated velocity to the mobile base
6. Rinse and repeat

(C)2016 Roi Yehoshua

Local Planner Parameters

- The file **base_local_planner.yaml** contains a large number of ROS Parameters that can be set to customize the behavior of the base local planner
- Grouped into several categories:
 - robot configuration
 - **goal tolerance**
 - forward simulation
 - trajectory scoring
 - oscillation prevention
 - global plan

(C)2016 Roi Yehoshua

Map Grid

- In order **to score trajectories efficiently**, a Map Grid is used.
- For each control cycle, a grid is created around the robot (the size of the local costmap), and the global path is mapped onto this area.
- This means **certain of the grid cells will be marked with distance 0 to a path point, and distance 0 to the goal.**
- A propagation algorithm then efficiently **marks all other cells with their Manhattan distance to the closest of the points marked with zero.**

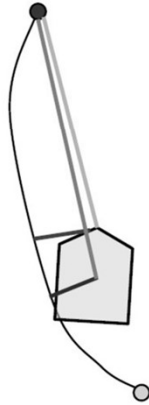
(C)2013 Roi Yehoshua

Oscillation Suppression

- Oscillation occur when in either of the x, y, or theta dimensions, positive and negative values are chosen consecutively.
- To prevent oscillations, when the robot moves in any direction, for the next cycles the opposite direction is marked invalid, until the robot has moved beyond a certain distance from the position where the flag was set.

(C)2013 Roi Yehoshua

Scoring Trajectories



Weighted Sum =

oscillation_cost
 + costmap_cost
 + goal_distance_cost
 + path_distance_cost
 + goal_alignment_cost
 + path_alignment_cost

(C)2016 Roi Yehoshua

Costmap

- A data structure that **represents places that are safe** for the robot to be in a grid of cells
- It is **based on the occupancy grid map** of the environment and **user specified inflation radius**
- There are two types of costmaps in ROS:
 - **Global costmap** is used for global navigation
 - **Local costmap** is used for local navigation
- **Each cell in the costmap has an integer value in the range [0 (FREE_SPACE), 255 (UNKNOWN)]**
- Managed by the costmap 2d package

(C)2016 Roi Yehoshua

Costmap Example

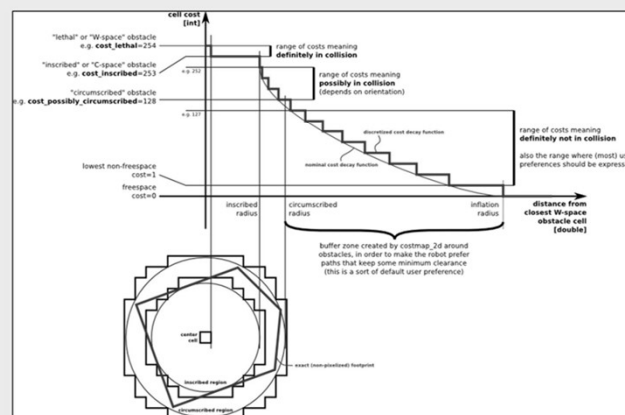


Taken from ROS Wiki http://wiki.ros.org/costmap_2d

(C)2016 Roi Yehoshua

Inflation

- Inflation is the process of propagating cost values out from occupied cells that decrease with distance



(C)2016 Roi Yehoshua

Map Updates

- The **costmap performs map update cycles** at the rate specified by the **update_frequency** parameter
- In each cycle:
 - **sensor data comes in**
 - **marking and clearing operations are performed** in the underlying occupancy structure of the costmap
 - this **structure is projected into the costmap** where the appropriate cost values are assigned as described above
 - **obstacle inflation is performed** on each cell with a LETHAL_OBSTACLE value
 - This consists of propagating cost values outwards from each occupied cell out to a user-specified inflation radius

(C)2016 Roi Yehoshua

Costmap Parameters Files

- Configuration of the costmaps consists of three files:
 - costmap_common_params.yaml
 - global_costmap_params.yaml
 - local_costmap_params.yaml
- http://wiki.ros.org/costmap_2d/hydro/obstacles

(C)2016 Roi Yehoshua

move_base

- The move_base package lets you **move a robot to desired positions** using the navigation stack
- The move_base node **links together a global and local planner** to accomplish its navigation task
- It may **optionally perform recovery behaviors** when the robot perceives itself as stuck

(C)2016 Roi Yehoshua

P3AT Navigation

- Navigation package includes demos of map building using gmapping and localization with amcl, while running the navigation stack
- In **nav_params** subdirectory it contains configuration files for P3AT navigation
- Sizin uygulamanızda **nav_params** klasöründeki parametreler değiştirilerek testler yapılacaktır.

(C)2016 Roi Yehoshua

Navigation Configuration Files

Configuration File	Description
global_planner_params.yaml	global planner configuration
navfn_global_planner_params.yaml	navfn configuration
dwa_local_planner_params.yaml	local planner configuration
costmap_common_params.yaml global_costmap_params.yaml local_costmap_params.yaml	costmap configuration files
move_base_params.yaml	move base configuration
amcl.launch.xml	amcl configuration

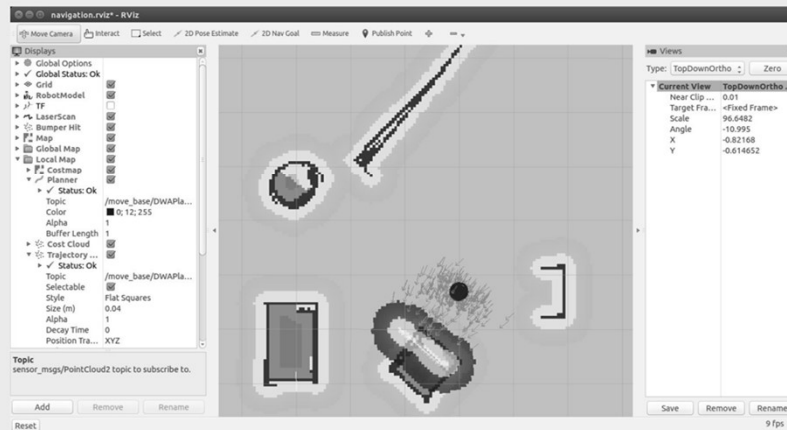
(C)2016 Roi Yehoshua

rviz with Navigation Stack

- rviz allows you to:
 - Provide an approximate location of the robot (when starting up, the robot doesn't know where it is)
 - Send goals to the navigation stack
 - Display all the visualization information relevant to the navigation (planned path, costmap, etc.)

(C)2016 Roi Yehoshua

rviz with Navigation Stack



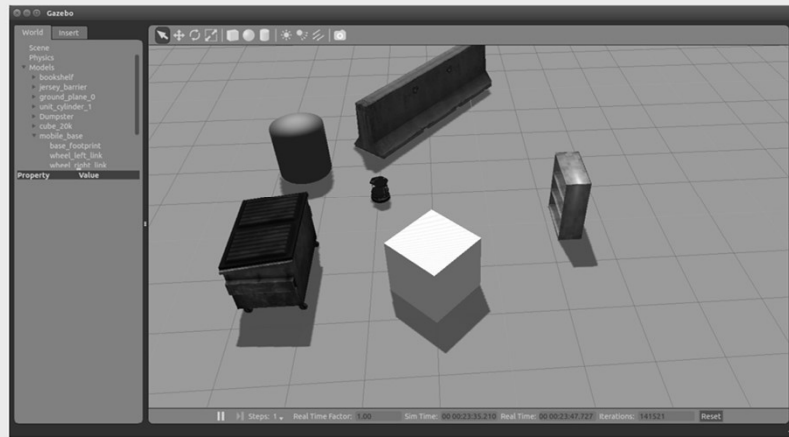
(C)2016 Roi Yehoshua

Localize the Robot

- When starting up the robot doesn't know where it is
- For example, let's move the robot in Gazebo to (-1,-2)
- Now to provide it its approximate location on the map:
 - Click the "2D Pose Estimate" button
 - Click on the map where the robot approximately is and drag in the direction the robot is pointing
- You will see a collection of arrows which are hypotheses of the position of the robot
- The laser scan should line up approximately with the walls in the map
 - If things don't line up well you can repeat the procedure

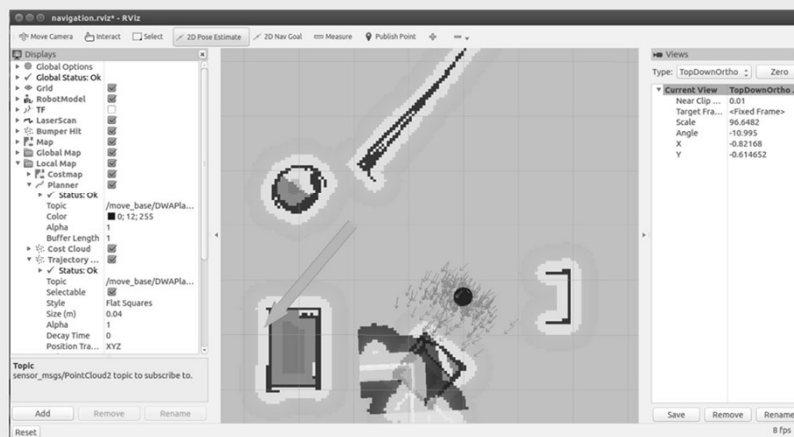
(C)2016 Roi Yehoshua

Localize the Robot



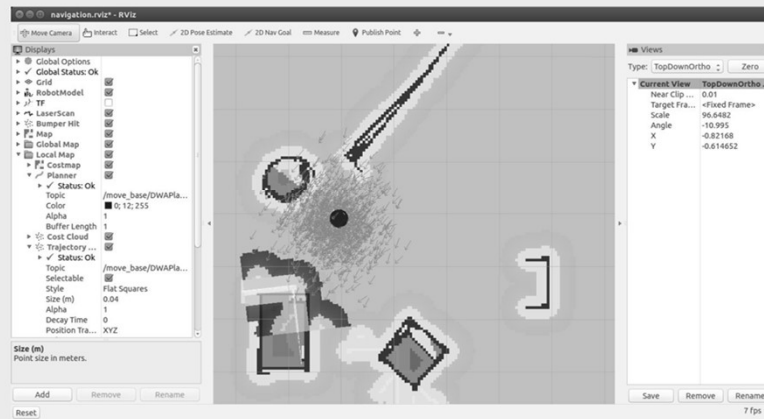
(C)2016 Roi Yehoshua

Localize the Robot



(C)2016 Roi Yehoshua

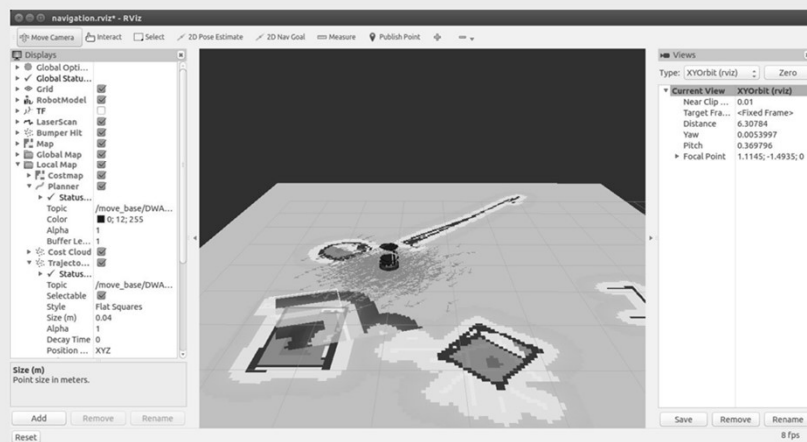
Localize the Robot



(C)2016 Roi Yehoshua

Localize the Robot

- You can change the current view (on right panel):



(C)2016 Roi Yehoshua

Particle Cloud in rviz

- The **Particle Cloud** display shows the particle cloud used by the robot's localization system
- The spread of the cloud represents the localization system's uncertainty about the robot's pose
- As the robot moves about the environment, this cloud should shrink in size as additional scan data allows amcl to refine its estimate of the robot's position and orientation
- **To watch the particle cloud in rviz:**
 - Click Add Display and choose Pose Array
 - Set topic name to /particlecloud

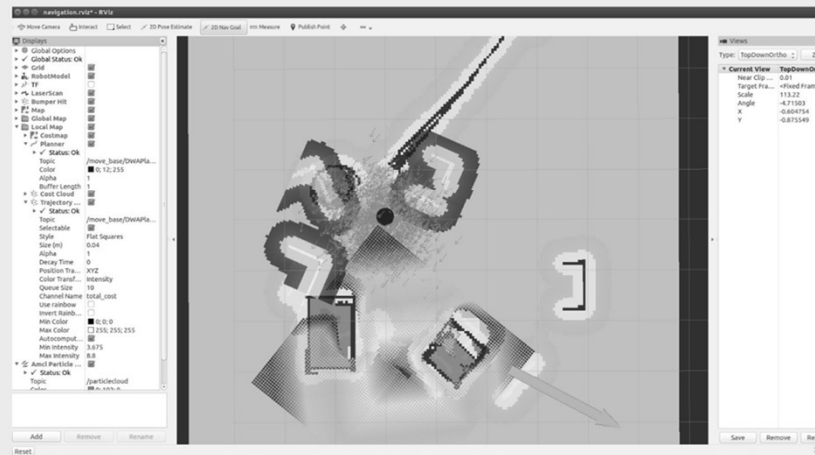
(C)2016 Roi Yehoshua

Send a Navigation Goal

- With the robot localized, it can then autonomously plan through the environment
- To send a goal:
 - Click the "**2D Nav Goal**" button
 - Click on the map where you want the robot to drive and drag in the direction where it should be pointing at the end
- If you want to stop the robot before it reaches it's goal, send it a goal at it's current location

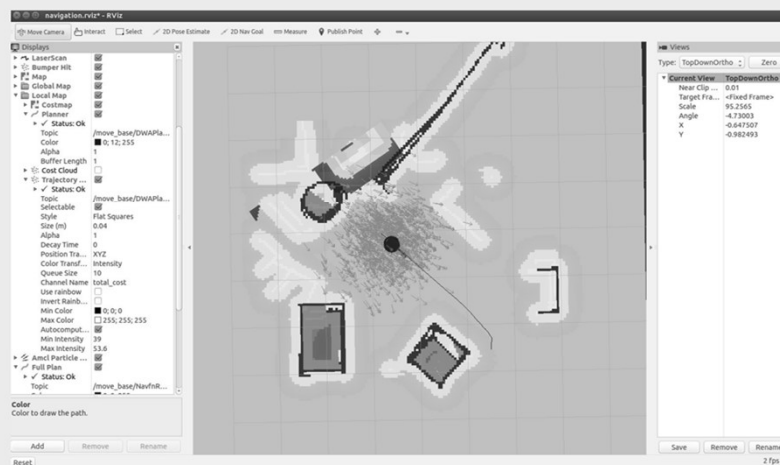
(C)2016 Roi Yehoshua

Send a Navigation Goal



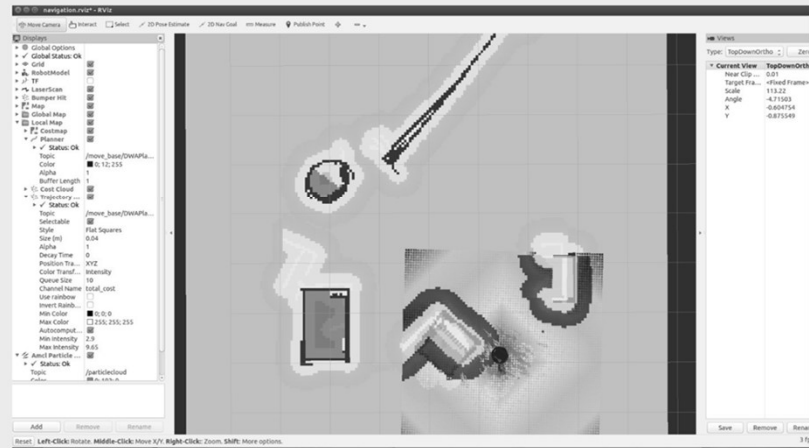
(C)2016 Roi Yehoshua

Robot Moves to Destination



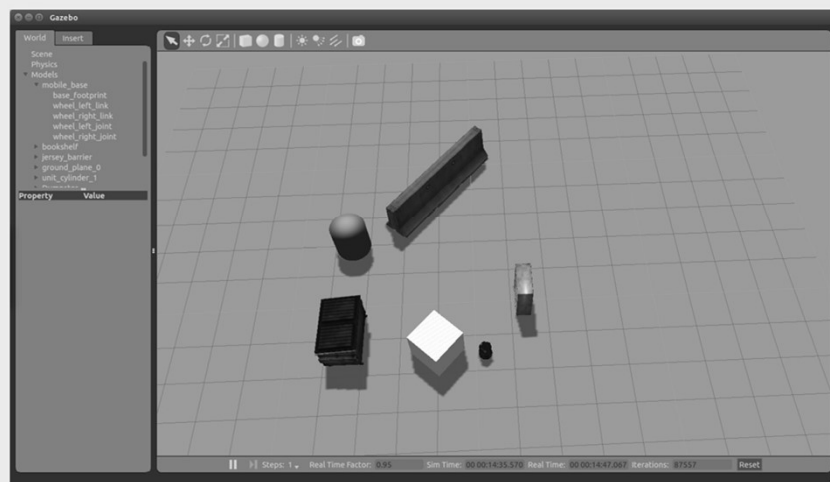
(C)2016 Roi Yehoshua

Final Pose



(C)2016 Roi Yehoshua

Final Pose In Gazebo



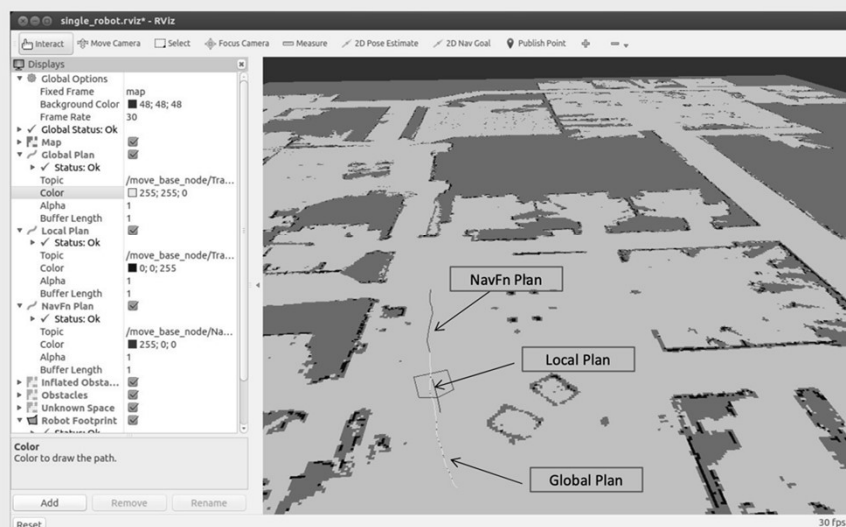
(C)2016 Roi Yehoshua

Navigation Plans in rviz

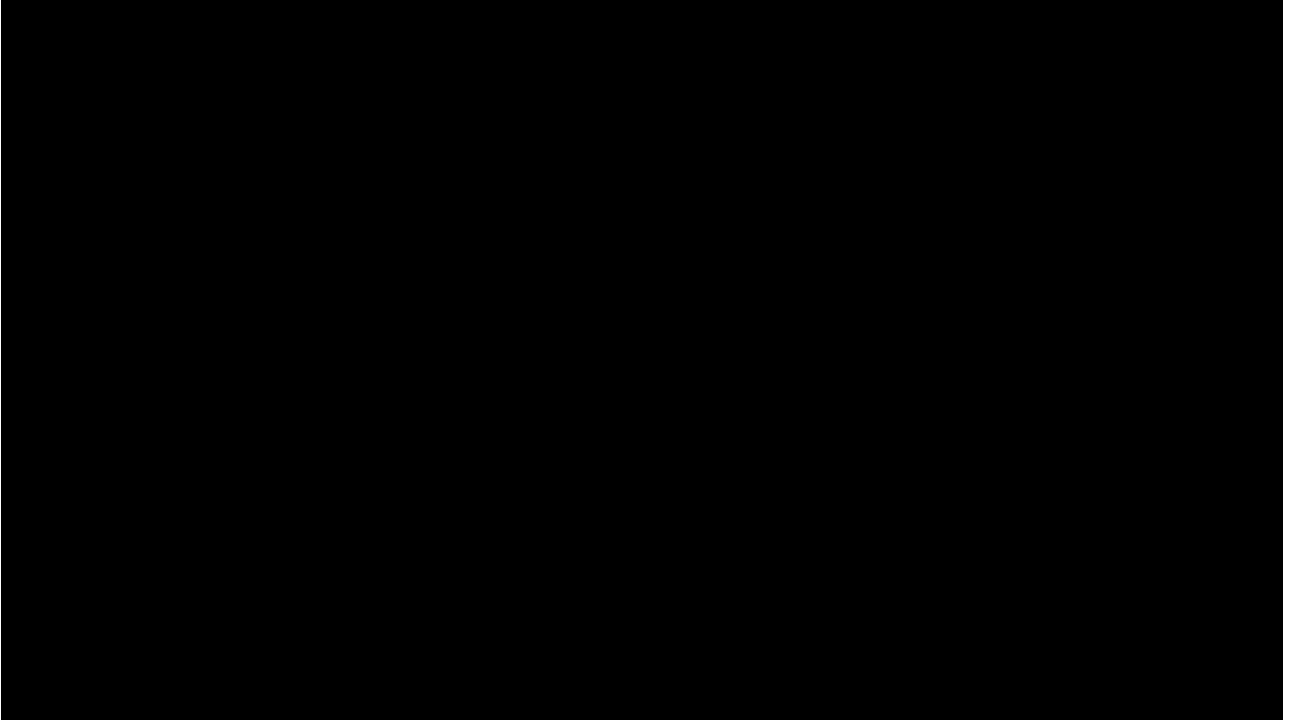
- **NavFn Plan**
 - Displays the **full plan** for the robot computed by the global planner
 - Topic: /move_base_node/NavfnROS/plan
- **Global Plan**
 - Shows the **portion of the global plan** that the local planner is currently pursuing
 - Topic: /move_base_node/TrajectoryPlannerROS/global_plan
- **Local Plan**
 - Shows **the trajectory associated with the velocity commands** currently being commanded to the base by the local planner
 - Topic: /move_base_node/TrajectoryPlannerROS/local_plan

(C)2016 Roi Yehoshua

Navigation Plans in rviz



(C)2016 Roi Yehoshua



Sabırla Dinlediğiniz İçin Teşekkürler

BLM4830
Robot
Teknolojisine
Giriş
Hafta 7

